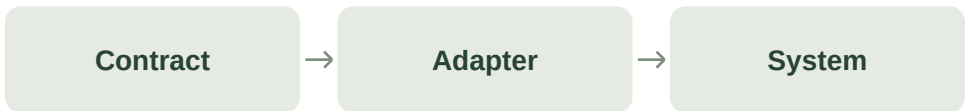


Design Patterns

Make the boundary visible

A pattern is useful when it makes a recurring design problem easier to understand. It is not a requirement to put every application behind layers of indirection.



For a PDF reader, a small interface for storing library metadata lets the application service work with a disk implementation or a test double.

A new engineer should know what a module owns before reading its implementation.

One reason to change

Keep rendering, workflow decisions and durable storage separate. A typography change should not require editing a file-import transaction.

Small files help, but file length alone does not guarantee good boundaries. A module should expose a coherent contract and avoid leaking details that callers do not need.

Try naming a module without using the words manager, helper or utils.

Depend on a contract

An application workflow can depend on a storage contract instead of constructing a specific database or file store. The composition root supplies the concrete implementation.

```
protocol LibraryPersistence {  
    func load() throws -> Library  
    func save(_ library: Library) throws  
}
```

This keeps tests focused. A storage failure can be introduced deliberately without filling the device or corrupting real user files.

Test that an unsuccessful save does not leak uncommitted in-memory state.

Use patterns sparingly

Every abstraction introduces a concept that the next engineer must learn. Add the boundary when it protects a responsibility, enables an important test or isolates a volatile dependency.

Keep the simplest implementation that meets the real constraints. An offline personal library does not need a distributed backend to look or feel well made.

This sample explains the design approach used in Shelf. The architecture and decision records in the source package describe the actual implementation.

What can you remove without weakening a guarantee the user depends on?